МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ
УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра "Програмна інженерія"

ЗВІТ

з лабораторної роботи №2
з дисципліни "Основи Програмної Інженерії"

Виконали
ст. гр. ПЗПІ-25-3
Стогній.Є.Д

Прийняв:
В.Гребенюк

Харків 2026

ЛАБОРАТОРНА РОБОТА № 2

МОДЕЛЮВАННЯ ПРОГРАМНОЇ СИСТЕМИ ЗАСОБАМИ UML (UNIFIED MODELING LANGUAGE)

2.1 Мета роботи

Навчитися трансформувати функціональні вимоги до програмного забезпечення у візуальні моделі UML. Оволодіти інструментарієм проектування структури та динаміки системи (Use Case, Class, Sequence diagrams). Продемонструвати навички аналізу предметної області та побудови матриць відповідності для контролю покриття вимог проєктними рішеннями.

2.2 Хід роботи

2.2.1 Предметна область

DeltaPhys - платформа для пошуку партнерів та професійного нетворкінгу в екстремальних видах спорту.

2.2.2 Функціональні вимоги

FR-01: Модуль автентифікації повинен забезпечувати реєстрацію нових користувачів через електронну пошту.
FR-02: Функціонал системи має дозволяти формування пошукових запитів у вигляді анкет-сесій для нетворкінгу.
FR-03: Система повинна генерувати публічний шар карти з геомітками запланованих спортивних подій.
FR-04: Програмне забезпечення має надавати статус «верифіковано» користувачам після перевірки їхніх даних.
FR-05: У системі має бути реалізована можливість обміну текстовими повідомленнями між учасниками конкретного виїзду.
FR-06: Система повинна дозволяти фільтрацію подій за видом спорту

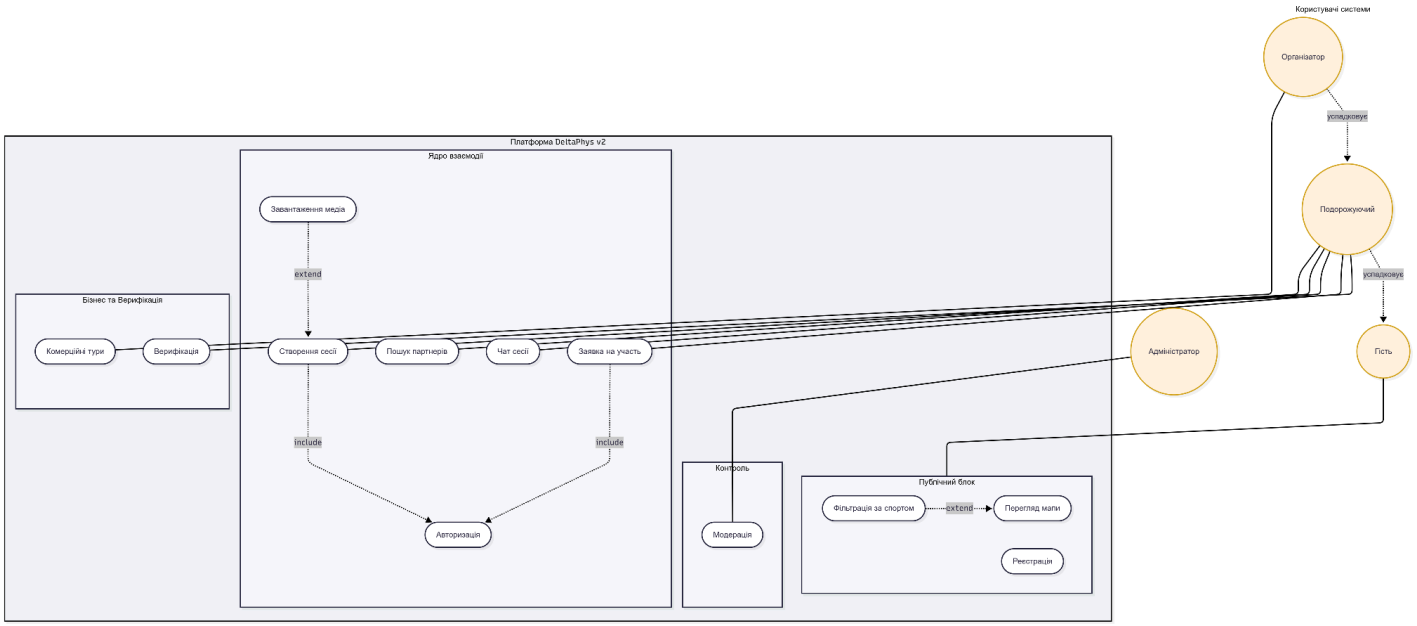
2.2.3 Діаграма прецедентів

Код

```
flowchart TB
    classDef cluster_box fill:#f8f9fa,stroke:#333,stroke-width:1px,stroke-dasharray: 5 5;
    classDef user_node fill:#fff4dd,stroke:#d4a017,stroke-width:2px;
    classDef system_node fill:#e3f2fd,stroke:#1565c0,stroke-width:2px;

    subgraph Users [Користувачі системи]
        direction LR
        G((Гість)):::user_node
        T((Подорожуючий)):::user_node
        O((Організатор)):::user_node
    end

    subgraph DeltaPhys [Платформа DeltaPhys v2]

        subgraph Public [Публічний блок]
            UC1([Перегляд мапи])
            UC11([Фільтрація за спортом])
            UC2([Реєстрація])
            UC11 -.->|extend| UC1
        end

        subgraph Core [Ядро взаємодії]
            UC3([Пошук партнерів])
            UC4([Чат сесії])
            UC5([Створення сесії])
            UC6([Заявка на участь])
            UC_Auth([Авторизація])
            UC_Upload([Завантаження медіа])

            UC5 -.->|include| UC_Auth
            UC6 -.->|include| UC_Auth
            UC_Upload -.->|extend| UC5
        end

        subgraph Services [Бізнес та Верифікація]
            UC7([Верифікація])
            UC8([Комерційні тури])
        end

        subgraph AdminBlock [Контроль]
            UC10([Модерація])
        end
    end
```

```
A((Адміністратор)):::user_node

G --- Public

T --- UC3
T --- UC4
T --- UC5
T --- UC6
T --- UC7

O --- UC8
A --- UC10

T -.->|успадковує| G
O -.->|успадковує| T

style Users fill:none,stroke:none
style DeltaPhys fill:#f0f4f8,stroke:#000,stroke-width:2px
```

Діаграма



Виконав: Стогній

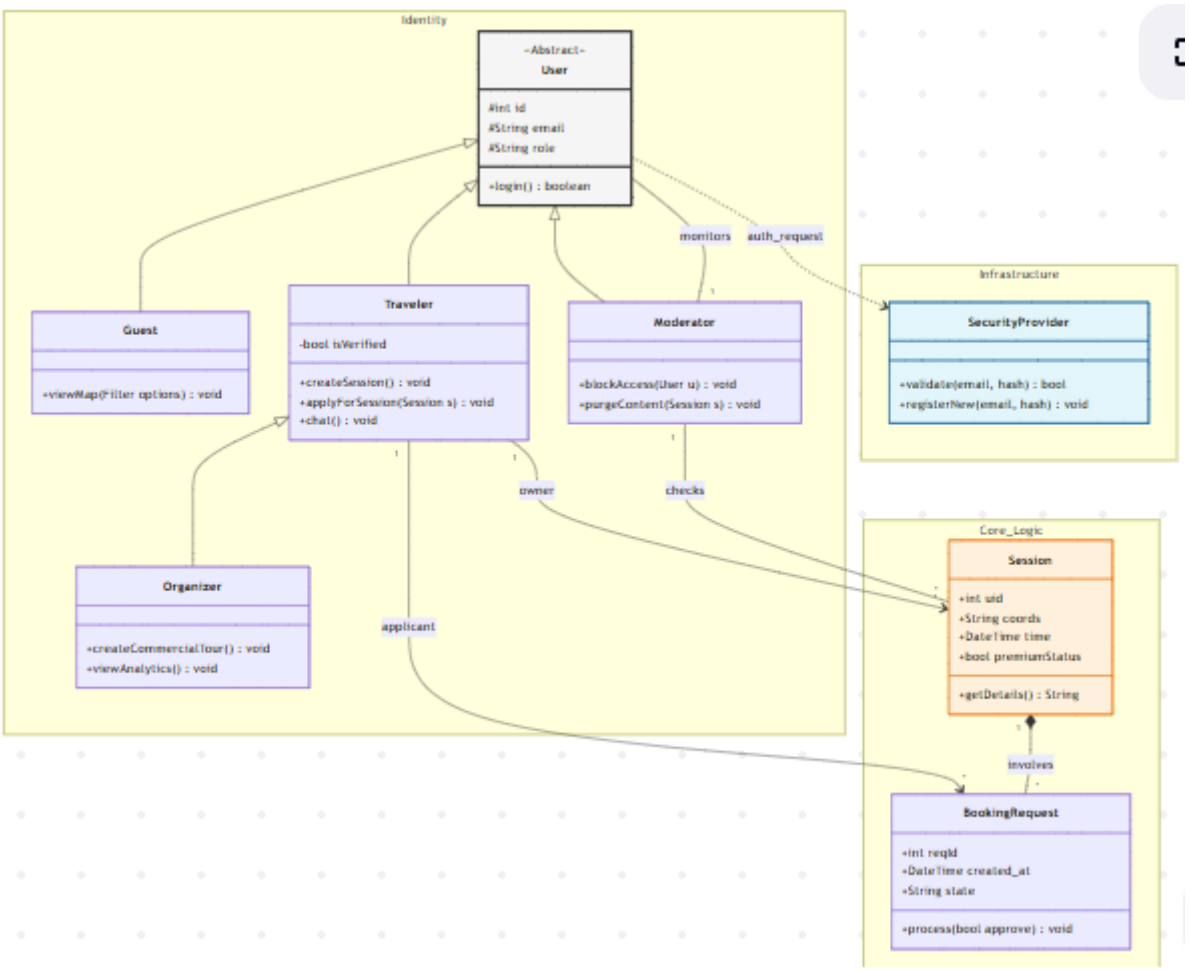
2.2.4 Діаграма класів

Код

```
classDiagram
    namespace Identity {
        class User {
            <<Abstract>>
            #int id
            #String email
            #String role
            +login() boolean
        }
        class Guest {
            +viewMap(Filter options) void
        }
        class Traveler {
            -bool isVerified
            +createSession() void
            +applyForSession(Session s) void
            +chat() void
        }
        class Organizer {
            +createCommercialTour() void
            +viewAnalytics() void
        }
        class Moderator {
            +blockAccess(User u) void
            +purgeContent(Session s) void
        }
    }

    namespace Core_Logic {
        class Session {
            +int uid
            +String coords
            +DateTime time
            +bool premiumStatus
            +getDetails() String
        }
        class BookingRequest {
            +int reqId
            +DateTime created_at
            +String state
            +process(bool approve) void
        }
```

```
}

namespace Infrastructure {
    class SecurityProvider {
        +validate(email, hash) bool
        +registerNew(email, hash) void
    }
}

User <|-- Guest
User <|-- Traveler
User <|-- Moderator
Traveler <|-- Organizer

User ..> SecurityProvider : auth_request

Traveler "1" --> "*" Session : owner
Traveler "1" --> "*" BookingRequest : applicant

Session "1" *-- "*" BookingRequest : involves

Moderator "1" -- "*" User : monitors
Moderator "1" -- "*" Session : checks

style User fill:#f9f9f9,stroke:#333,stroke-width:2px
style SecurityProvider fill:#e1f5fe,stroke:#01579b
style Session fill:#fff3e0,stroke:#ef6c00
```

Діаграма



Виконав: Стогній

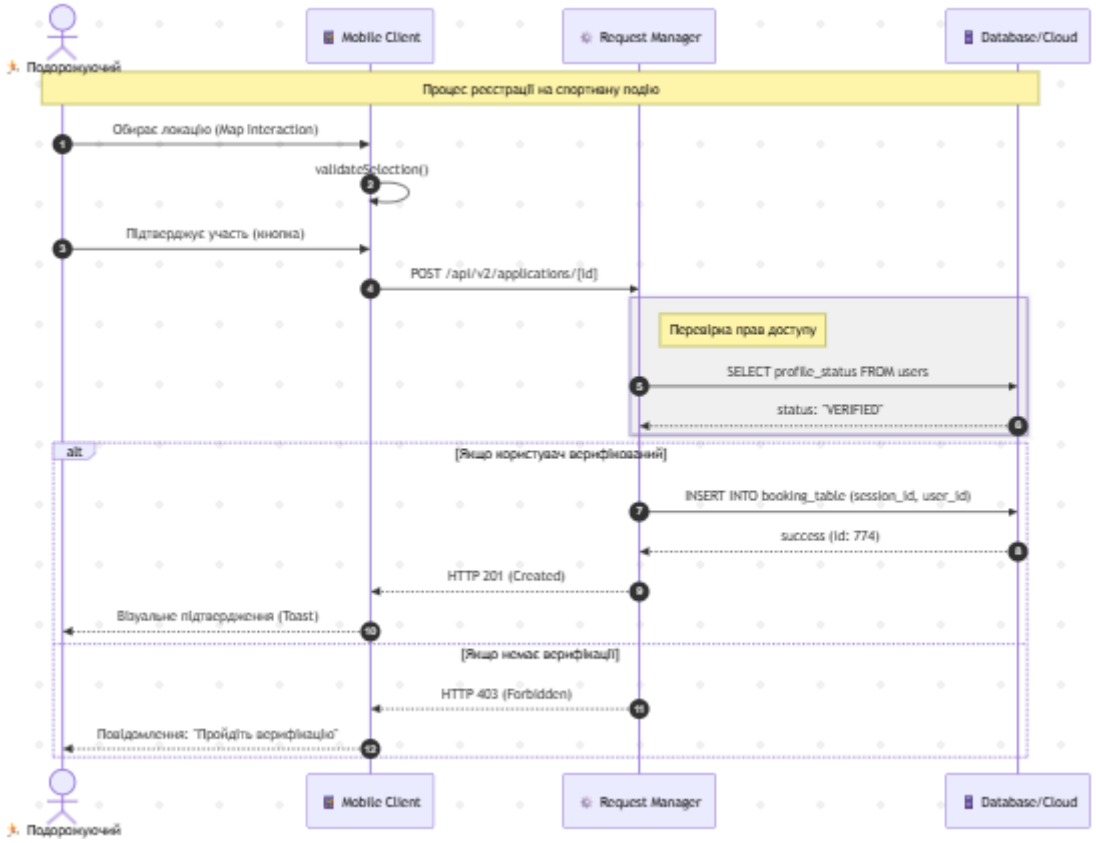
2.2.5 Діаграма послідовності

Код

```
sequenceDiagram
    autonumber
    actor User as 🏃 Подорожуючий
    participant App as 📱 Mobile Client
    participant Controller as ⚙️ Request Manager
    participant Store as 🗄️ Database/Cloud

    Note over User, Store: Процес реєстрації на спортивну подію

    User->>App: Обирає локацію (Map Interaction)
    App->>App: validateSelection()

    User->>App: Підтверджує участь (кнопка)

    App->>Controller: POST /api/v2/applications/{id}

    rect rgb(240, 240, 240)
```

```
    Note right of Controller: Перевірка прав доступу
    Controller->>Store: SELECT profile_status FROM users
    Store-->>Controller: status: "VERIFIED"
end

alt Якщо користувач верифікований
    Controller->>Store: INSERT INTO booking_table (session_id, user_id)
    Store-->>Controller: success (id: 774)
    Controller-->>App: HTTP 201 (Created)
    App-->>User: Візуальне підтвердження (Toast)
else Якщо немає верифікації
    Controller-->>App: HTTP 403 (Forbidden)
    App-->>User: Повідомлення: "Пройдіть верифікацію"
end
```

Діаграма



Виконав: Стогній

2.2.6 Матриця трасовності

Вимога	Use Case	Класи	Sequence
FR-01: Реєстрація через email	UC2: Реєстрація	AuthService, User	
FR-02: Пошукові запити (анкети)	UC5: Створення сесії	Traveler, Session	
FR-03: Публічна карта	UC1: Перегляд мапи	Guest, Session	
FR-04: Статус «верифіковано»	UC7: Верифікація	Traveler (атрибут `statusVerified`)	Перевірка `statusVerified`
FR-05: Обмін повідомленнями	UC4: Чат сесії	Traveler (метод `sendMessage`)	
FR-06: Фільтрація за спортом	UC11: Фільтрація	`Guest` (метод `showMapWithFilters`)	showMapWithFilters()